

Proyecto #1 - Administrador de Diccionario de Datos StreamUCV

Integrantes del Grupo:

- [Gustavo Berne 30.188.117]
 - [Patricia Cibeira 25.869.022]
 - [Alejandro González 28.313.390]
-

1. Descripción General de la Solución

Para resolver la problemática planteada por el departamento de Análisis de Datos de **StreamUCV**, se desarrolló una aplicación de escritorio interactiva utilizando **Python** y su librería estándar para interfaces gráficas, **tkinter**.

El objetivo principal de la aplicación es brindar visibilidad estructurada sobre el estado interno de la base de datos **StreamUCV**, específicamente en el esquema **streaming**. La solución actúa como un puente amigable entre los usuarios y el motor de base de datos, permitiendo ejecutar 10 reportes técnicos fundamentales sin necesidad de que el analista posea conocimientos avanzados de consultas SQL.

Arquitectura de la Aplicación: La aplicación está consolidada en un único archivo fuente (`main.py`) para facilitar su distribución y configuración. Consta de dos componentes lógicos principales:

1. **Módulo de Conexión y Datos (`DatabaseHandler`):** Administra la conexión a Microsoft SQL Server utilizando el driver **ODBC Driver 17 for SQL Server** y la librería `pypyodbc`. Esta clase contiene la lógica para interrogar dinámicamente el Diccionario de Datos de SQL Server y devolver estructuras de datos nativas de Python.
 2. **Interfaz Gráfica de Usuario (`AppGUI`):** Implementada con `tkinter` y `ttk`, proporciona un menú compuesto por 10 botones de acción. Los resultados de cada reporte se despliegan de forma ordenada y legible mediante un widget `Treeview` (tabla dinámica con barras de desplazamiento).
-

2. Explicación del Uso del Diccionario de Datos

Para cumplir con los requerimientos funcionales, la aplicación realiza consultas directas a las vistas del sistema (System Catalog Views) y vistas dinámicas de administración (DMVs) que conforman el Diccionario de Datos de Microsoft SQL Server:

1. **Tablas e Índices (Req. 1 y 2):**
 - Se utilizaron las vistas `sys.tables`, `sys.indexes` y `sys.schemas`. Se filtraron los resultados usando `WHERE s.name = 'streaming'` para enfocar la búsqueda en el esquema requerido y `i.type > 0` para excluir *heaps* (montículos

sin índice).

2. Restricciones (Req. 3):

- Se consultó `sys.objects`, filtrando por el esquema pertinente y por los tipos de restricción específicos (Check, Default, Foreign Key, Primary Key, Unique) mediante `type IN ('C', 'D', 'F', 'PK', 'UQ')`.

3. Detalles de Índices (Req. 4):

- Se cruzaron las vistas `sys.indexes`, `sys.index_columns`, `sys.columns` y `sys.tables` para identificar qué columnas componen cada índice y si estos poseen propiedades de unicidad (`is_unique`).

4. Triggers (Req. 5):

- Se interrogó la vista `sys.triggers` unida a `sys.tables` para extraer la metadata de los disparadores, su estado de activación (`is_disabled`) y las tablas desencadenantes.

5. Cálculos de Almacenamiento (Req. 6, 7, 8 y 9):

- Para estimar el tamaño en bytes de cada columna (Req. 8), se extrajo la propiedad `max_length` cruzando `sys.columns` con `sys.types`. La suma de estos tamaños máximos permitió calcular la estimación del **tamaño del registro** (Req. 7).
- Para obtener el tamaño total ocupado por tabla (Req. 6) se utilizó la vista dinámica `sys.dm_db_partition_stats` (`SUM(used_page_count) * 8.0`).
- El **factor de bloqueo** (Req. 9) fue calculado programáticamente dividiendo el tamaño estándar de una página de datos de SQL Server (8192 bytes) entre la sumatoria del tamaño del registro, obteniendo la cantidad de registros que caben por bloque.

6. Costo de Consulta (Req. 10):

- Se programó un simulador de plan de ejecución básico. La aplicación verifica en `sys.index_columns` si la columna dada posee un índice (donde la columna sea el nivel raíz `key_ordinal = 1`).
- Si no existe índice, el costo refleja un *Full Table Scan*, equivalente a leer todas las páginas de datos de la tabla (`p.index_id IN (0, 1)`).
- Si existe índice, se asume un costo optimizado (1 lectura de índice + 1 lectura a la página de datos), permitiendo estimar el impacto de tiempo basado en una transferencia teórica de 17 MB/s.

3. Instrucciones de Ejecución

Prerrequisitos

- Tener instalado **Python 3.x** en el equipo.
- Contar con la base de datos **StreamUCV** creada y cargada en un servidor local de Microsoft

SQL Server utilizando los scripts proporcionados.

Paso 1: Configurar la Conexión a la Base de Datos

1. Abrir el archivo `main.py` con cualquier editor de texto o IDE.
2. En las primeras líneas del archivo se encuentra la sección claramente identificable **"CONFIGURACIÓN DE CONEXIÓN A LA BASE DE DATOS"**.
3. Reemplace los valores de las variables según su entorno local.
 - *Nota:* Si utiliza autenticación de Windows en SQL Server, deje las variables `USERNAME` y `PASSWORD` con valores vacíos (`""`).

Paso 2: Instalación de Dependencias

Abra una terminal o línea de comandos, ubíquese en la carpeta del proyecto y ejecute el siguiente comando para instalar las librerías necesarias:

```
pip install -r requirements.txt
```

(Este comando instalará principalmente `pypyodbc`, un driver nativo de Python para ODBC que no requiere compilación).

Paso 3: Ejecutar la Aplicación

Existen dos alternativas para lanzar la aplicación:

Opción A: A través de código fuente (Terminal) Desde la misma terminal, inicie el programa ejecutando:

```
python main.py
```

Opción B: A través del archivo ejecutable (Recomendado) Para facilitar el despliegue, se ha generado un archivo ejecutable autónomo.

1. Navegue a la carpeta `dist/` dentro del directorio del proyecto.
2. Haga doble clic en el archivo `main.exe` (o el nombre que se le haya asignado).
3. La aplicación gráfica se abrirá de forma nativa sin necesidad de abrir consolas de comandos.

Aparecerá la interfaz gráfica principal con el menú de 10 botones. Simplemente haga clic en el reporte que desea visualizar para interactuar con la base de datos y ver los resultados en la tabla inferior.